

目錄

1. INTRODUCTION	2
A. BACKGROUND.....	2
B. PROBLEM DESCRIPTION	2
C. TECHNICAL CHALLENGE.....	3
D. OUR OBSERVATIONS	5
E. HOW THESE OBSERVATIONS HELP US DEVELOP THE PROPOSED METHOD.....	6
F. A SUMMARY OF EXPERIMENTAL RESULTS	7
2. RELATED WORK.....	9
A. PROMPT INJECTION ATTACKS IN CODE GENERATION	9
1) <i>Indirect Prompt Injection Attack</i>	9
2) <i>In code generation scenarios</i>	10
B. DEFENSES AGAINST PROMPT INJECTION	10
3. BACKGROUND KNOWLEDGE	12
4. THREAT MODEL	13
A. PROBLEM SETTING	14
B. DEFENDER’S CAPABILITY	16
5. PROPOSED METHOD.....	17
6. EVALUATION.....	22
A. SETUP	22
i. <i>Datasets</i>	22
ii. <i>Models</i>	22
iii. <i>Metrics</i>	23
iv. <i>Competitors</i>	24
v. <i>Hyperparameter Setting</i>	24
B. RESULT	26
i. <i>Defense on 6 Attack type compared with 3 baselines</i>	26
ii. <i>AUC-ROC</i>	27
iii. <i>Cumulative layer’s performance</i>	28
iv. <i>cross-model generalization</i>	29
C. ROBUSTNESS ANALYSIS.....	31
1. <i>Adaptive Attack</i>	31
2. <i>Transfer Attack</i>	33

7. CONCLUSION.....	36
REFERENCE.....	38

1. Introduction

A. Background

近年來，隨著 Code Large Language Model (LLM) 的普及，LLM 在軟體開發過程中已成為不可或缺的一部分，然而，訓練過後的 LLM 有指令遵循的能力，這意味著 LLM 會盡力地去滿足 user 的需求，這種特性導致了一種稱作 Prompt Injection Attack 的安全漏洞發生，攻擊者可以利用輸入的惡意提示詞 (prompt)，讓模型忽略系統原有的安全指令，轉而去執行攻擊者提供的惡意指令，例如: 駭客可以下 prompt 要求 LLM 寫一個程式 將公司的機密資料上傳到某個網站上，那因為 LLM 會滿足 user 的需求，因此就會產生這個惡意的程式碼

B. Problem Description

然而，Prompt Injection Attack 又分為 Direct 和 Indirect 的 Prompt Injection Attack，而 Direct 的意思是攻擊者就是 User，會直接在對話框中輸入惡意指令並餵給 model，而 Indirect Prompt Injection 則是使用者本身無惡意，但攻擊者會將惡意的 payloads 藏在 LLM 所檢索到的外部資料中，例如 GitHub 平台上的程式碼內，而由於 LLM 在處理輸入時，他會把使用者指令和外部資料都變成一長串的文字序列，一起餵給 LLM，所以當開發者或自動化工具將這些受污染的外部程式碼作為上下文檢索並餵給 LLM 時，

LLM 無法分辨哪些是上下文資料，哪些是應執行指令，導致 LLM 將檢索到的惡意 payload 視為高優先級的指令執行，進而產生攻擊者預期的惡意程式碼。

而這種安全漏洞將導致嚴重的安全性威脅。由於被誤導的 LLM 會生成隱含惡意邏輯的程式碼，如果開發者在本地執行這些惡意程式碼，攻擊者甚至能夠取得開發環境的控制權，進一步地去竊取 SSH Keys，公司機密資料，導致敏感資訊外洩。因此，如何建立有效且即時性的防禦機制來防禦

Indirect Prompt Injection，已成為 LLM 安全性的核心挑戰。

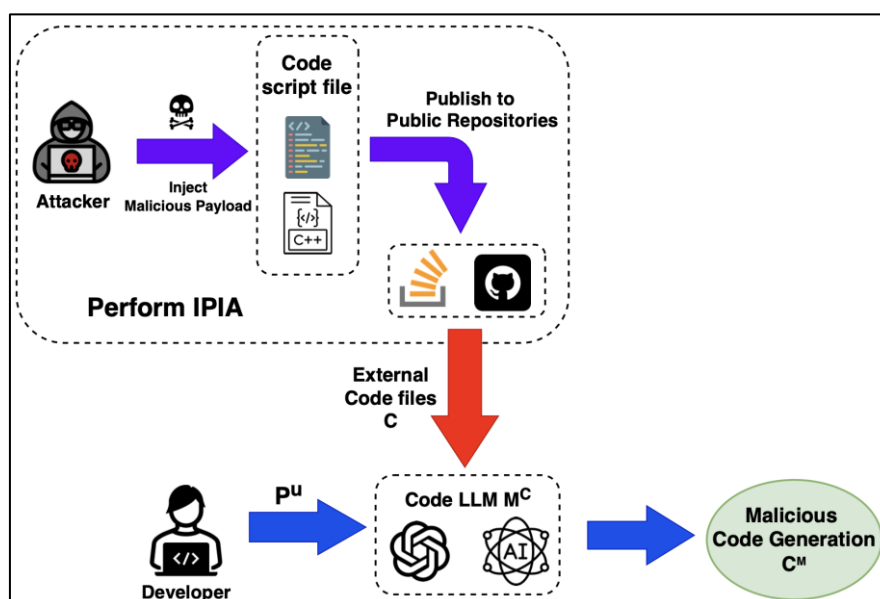


Figure 1 Problem Overview

C. Technical Challenge

儘管現有的防禦機制試圖透過輸入過濾，微調分類器，困惑度分析來防禦住 Prompt Injection Attack，然而，在面對專門針對 Code LLM 在程式碼任務下的新型攻擊時，仍然面臨著以下困境:

1. 困惑度分析和 Fine-tuning 過後的防禦偵測器 在語意特徵不明顯的新型攻擊上 偵測會失效，例如 **XOXO [1]** 以及 **ITGen [2]**，這些攻擊手法透過語意相同的變數名變換，在不改變程式碼執行邏輯的前提下隱藏惡意意圖，達成 Prompt Injection Attack，例如: 將 `USE_RAW_QUERIES` 這個變數名稱改成 `RAW_QUERIES`，而 LLM 生成的 code 就會從安全的變成含有 SQL Injection 的 code，然而開發者為了提升程式碼可讀性，經常會移除變數中冗餘的前綴或者簡化變數名稱，因此，對於分類器而言，`USE_RAW_QUERIES -> RAW_QUERIES` 看起來和正常的程式碼優化並無區別，這導致分類器難以區分這是惡意的上下文改動還是 正常的程式碼優化，進而造成漏判及誤判。

而且這些變數名稱在語法上是合法且看起來正常的，每個變數名的困惑度不會像亂碼一樣高，所以統計異常的困惑度偵測難以偵測出這種新型語意型攻擊
2. 新型對抗型攻擊例如 **ShadowCode [3]** 和 **INSEC [4]** 皆能在黑箱環境下對模型進行攻擊，黑箱環境是指攻擊者只能透過 API 查詢目標模型並觀察其輸出結果 (例如生成的程式碼)，而無法獲取目標模型的內部參數。但 ShadowCode 及 INSEC 的優化技術和 Token 空間的來源並不相同，ShadowCode 利用一個白箱的代理模型(Surrogate Model) 來計算梯度，在代理模型的 token 空間內優化出對抗性擾動，而 INSEC 則是引入了代

理 Tokenizer (Proxy Tokenizer) 來定義搜索空間，並透過基因演算法直接對目標模型進行迭代查詢以找出最佳攻擊字串，在這兩篇論文中提到，這兩種攻擊最終都會生成出一串能最大化模型輸出惡意 code 機率的亂碼型字串，並藏在註解中，以達成攻擊路徑。

而這些亂碼不具備明顯的語意特徵，由於目前的防禦框架難以窮舉所有的對抗性組合來去進行微調，也難以用意圖檢測去辨別，這讓現有的防禦框架難以防禦住 ShadowCode 以及 INSEC 這類動態生成惡意 payload 的攻擊

D. Our observations

在對現有的新型攻擊進行分析後，發現了以下現象，這些現象構成了我們防禦框架的基礎:

首先，我們觀察到 Prompt Injection Attack 的攻擊 payload 大多藏在程式碼的非執行區域，而由於修改這些區域不會影響程式碼的語法結構或執行邏輯，所以修改後編譯器無法察覺異狀，並且人工審查時這些非核心邏輯的區域也更容易被忽略掉，因此相比於修改程式碼的內部邏輯或核心演算法，針對這些區域的 Prompt Injection Attack 具有更高的隱蔽性。再來，這些區域的攻擊手法呈現出兩種不同的攻擊特徵:

1. 亂碼型的異常擾動 (例如 ShadowCode[3] 與 INSEC[4] 攻擊):

這類攻擊通常是透過黑箱優化技術來找出最佳的對抗性擾動，並塞入程

式碼的非執行區域中，而這些生成的字串往往呈現出無意義的亂碼型態，而對於 LLM 而言，這些不曾在 Training data 中出現過的異常字串會引發高困惑度，並在統計特徵上會有顯著的異常值

2. 語意型的隱蔽 trigger (XOXO[1]、ITGen[2]、Flashboom[5]攻擊):

這類 XOXO[1]、ITGen[2]攻擊透過變數重新命名等方式去尋找最佳的 payload 組合來作為攻擊 trigger，與上述的亂碼攻擊不同，這些 payload 在語法和拼寫上看起來是正常的 (例如使用常見單字)，因此，他們在統計上的困惑度較低，若是用異常檢測去偵測會有極高的漏判及誤判率。而另一種攻擊手法 Flashboom[5] 則是利用 LLM 的注意力分佈及推理資源有限的特性，來製造一些看似漏洞，實則正常的誘餌放在程式碼裡面，以此分散 LLM 的注意力，讓 LLM 忽略掉代碼中的真實漏洞。然而，這些攻擊皆具有一個關鍵特性:

因為這些 trigger 能讓模型生成惡意輸出，這代表他們對模型後續的生成輸出具有很高的影響力，若將該 trigger 進行移除或中性化，模型的最後一層 logits-layer 則會產生劇烈變化，而這顯示該 trigger 在 prompt injection attack 裡面扮演重要角色，因此高機率是惡意 trigger。

E. How these observations help us develop the proposed method

基於以上觀察，我們提出了一套 Hierarchical 的語法感知防禦框架，首先，因為攻擊 payload 皆隱藏在程式碼的非執行區域，所以先利用 Tree-sitter 進

行精確的 AST 解析，將偵測範圍收斂至最容易 embedding 攻擊 payload 的「非執行區域」，避免干擾程式碼的核心邏輯，而防禦架構分為三個階段：

第一層是低延遲的預過濾層，針對明顯的 Prompt Injection Attack 以及 Flashboom[5] 攻擊執行正則匹配與結構異常檢測，dead decoys 檢測，將明顯的 Prompt Injection Attack 以及 Flashboom 裡的誘餌(dead decoys)給偵測出來，針對亂碼的對抗型攻擊 (例如 ShadowCode[3] / INSEC[4]) 利用攻擊的 payload 會有統計異常的特性，透過 Dynamic Min-K%演算法作為第二層防禦，該演算法透過統計異常檢測的方式，計算文本中每個 token 的 negative log-likelihood，並聚焦於 k%最高 negative log-likelihood (最低機率)的 token，最終偵測出對抗性攻擊

接著，針對語意型的隱蔽攻擊 (XOXO[1], ITGen[2])，由於這類攻擊無法透過異常檢測，因此無法在前面兩層就被偵測出來，所以我們在第三層設計了 **CST-Guided Node Perturbation Analysis** 防禦框架，透過修改或刪除節點，並計算模型在修改前後的最後一層 logits-layer 的變化量是否劇烈，若移除看似正常的節點 (例如: 變數名稱 RAW_QUERIES) 後，模型的輸出 logits 產生劇烈變動，則判定該節點為惡意 trigger。

F. A summary of experimental results

結果顯示，我們的方法在偵測不同的新型 Indirect Prompt Injection Attack 時，(包括 XOXO, ITGen, Flashboom, ShadowCode, INSEC, CoTDeceptor 攻

擊) 平均的 F1 Score 為 0.817，高於 KillBadCode[6] (平均 f1 score 為 0.28)，DePA[7] (平均 f1 score 為 0.505) 及 CodeGarrison [8] (平均 f1 score 為 0.738) 而在偵測並清潔這些攻擊資料集後，減緩攻擊成功率(ASR)的結果也優於 DePA，例如 XOXO 攻擊可以從原本的 99.3%降到 11.1%，ShadowCode 攻擊則是從原本的 60%降到 22.4%

G. Conclusion:

總結來說，本文提出了一套 Hierarchical 的語法感知防禦框架，有效解決了 Code LLM 在面臨 Indirect Prompt Injection Attack 時構成的威脅，本研究的主要貢獻如下：

1. 基於程式碼語法感知的 Hierarchical 防禦框架:

跟以往將程式碼視為純文本的傳統方法不同，我們利用了 Tree-sitter 生成 CST 來精準鎖定註解，字串與變數名稱的非執行區域，將偵測範圍限縮在高風險區，並確保偵測清潔過程不會破壞程式碼的核心邏輯與功能，並且藉由三層防禦來防禦多種新型的 Prompt Injection Attack 例如 ITGen, Flashboom

2. 基於 CST 節點以及 Entropy 將 Min-K%演算法進行改良及優化:

導入 Dynamic K 值的概念，傳統的 Min-K%演算法通常將整段程式碼視為純文本，並採用固定的 K 值比例，由於在面對長度變化極大的程式碼節點時，容易因固定比例 K 而漏判隱藏在其中的短惡意 payload，或

在長的正常註解中產生誤判，為解決傳統異常檢測在程式碼領域的侷限性，導入了改良版的 CST-Guided 動態 Min-K%演算法

3. 提出創新的 CST-Guided Node Perturbation Analysis

本研究基於 **Differential Mutation Testing** 的概念 (觀察微小的輸入變化對輸出造成的差異)，提出了一套創新的 **CST-Guided Node Perturbation Analysis**，透過測量節點在移除或中性化前後對模型 logits-layer 的影響力，來識別出外表自然但具有惡意的隱蔽 trigger

2. Related Work

分為兩個面向：針對程式碼生成的 Prompt Injection Attacks，以及現有的防禦機制 Prompt Injection Defense

A. Prompt Injection Attacks in Code Generation

Prompt Injection Attack 是藉由惡意指令來去操控 LLM，使其產生惡意輸出。而早期研究主要集中於直接的越獄攻擊 (Direct Jailbreaking) [9]，透過精心設計的 template 讓模型繞過安全政策。然而，隨著 LLM 應用場景越來越廣泛，攻擊形式逐漸演變為更隱蔽的 Indirect Prompt Injection

1) Indirect Prompt Injection Attack

間接攻擊不會將惡意指令放入 user prompt 中餵給 LLM，而是將惡意指令注入到 LLM 檢索到的外部數據中。例如，[10] 透過污染 RAG (Retrieval-Augmented Generation) 知識庫來讓模型產生惡意輸出；[11] 則探討了在

multi-turn 對話中的上下文污染，[12] 則是針對 AI Agent 的場景下，攻擊者可以透過操縱環境輸入來操縱 AI Agent 的行為。

2) In code generation scenarios

在程式碼生成領域，攻擊者利用程式碼的特殊結構 (如註解，字串，變數名稱) 來將惡意 payload 隱藏起來，這使得攻擊更具隱蔽性。

- **注入 payload 的位置:**

[13] 利用 Git Commit Message 注入惡意 payload；ShadowCode [3] 則將攻擊藏在程式碼的非功能區域，例如 Dead Code 及註解中

- **新的攻擊手法:**

最新的研究展示了更複雜的攻擊模式，INSEC [4] 採用梯度優化方法生成對抗性後綴，並注入到註解中，這些後綴以亂碼形式呈現，能讓攻擊者操縱模型的生成行為。

而 XOXO [1] 以及 ITGen[2] 則針對程式碼場景，透過變數重命名來尋找最具攻擊性的變數名稱組合，由於 XOXO 及 ITGen 皆是合法且自然的變數名稱，因此傳統的困惑度檢測難以察覺。

Flashboom[5] 則是利用注入誘餌來分散模型注意力，讓模型忽略隱蔽的軟體漏洞

B. Defenses against Prompt Injection

面對以上攻擊，現有的 Prompt Injection Defense 研究主要集中於 NLP

(Natural Language Process)領域，很少有針對程式碼結構所設計的防禦框架

1) Perplexity-based Detection:

困惑度是檢測對抗性攻擊的常見指標，[14] 利用 PPL 過濾高困惑度的輸入；

[15] 則引入了 Min-K%方法，專注於檢測 token 機率最低的區段，在檢測異

常字串上具有顯著效果，然而，這類方法將程式碼視為純文本，容易在面對

XOXO 這類低困惑度的語意攻擊時失效，或是在面對語法複雜的正常程式碼

上產生出高誤報率，[7]則引入了 line-level token

2) Learning-based Defenses:

透過模型訓練來增強安全性，[16] 利用強化學習 (Reinforcement Learning) 來

進行安全對齊；[17] 和 [18] 則透過微調 (Fine-tuning) 來防禦特定的攻擊模

式，儘管有效，但這些訓練需要花費大量時間成本，且當新攻擊手法出來

時，又需要再重新訓練。

3) Limitations of Existing Approaches in Code Domain:

現有防禦大多針對 NLP 任務所設計，缺乏對程式碼語法結構的理解，而少

數針對程式碼的研究如 [6] 雖然專注在 Data Mitigating 部分，但方法主要仍

依賴於傳統的 PPL 檢測

4) Our Approach:

為了填補上述研究缺口，本文提出了基於 Tree-sitter 的三層防禦機制，與現

有方法不同，我們的方法不將程式碼視為純文本，而是利用 Tree-sitter 解析

程式碼結構，並精確定位高風險區域，再透過 CST-Guided Dynamic Min-K% 演算法以及 CST-Guided Node Perturbation Analysis 來防禦亂碼攻擊 (INSEC, ShadowCode) 以及語意隱蔽性攻擊 (XOXO, ITGen, Flashboom)

3. Background Knowledge

- **Indirect Prompt Injection Attack:** Indirect Prompt Injection Attack 是針對檢索外部資料來源 (例如 web, document, email, code) 的 LLM 去進行惡意輸出的一種攻擊，攻擊者不會直接在對話框輸入惡意指令，而是將惡意指令藏在模型會讀取的外部資料中，當模型處理這些外部資料時，惡意指令便會讓模型忽略原本的指令，轉而去執行惡意輸出。這種攻擊隱蔽性非常高，是目前 RAG 和 AI Agent 架構面臨的主要安全威脅之一
- **AST:** 將程式碼語法用樹狀結構表示，能夠精確定位程式的語法結構。在 AST 的結構中，程式碼的每一個基本語法單元都會被表示為一個節點 (Node)。例如: 對於一行程式碼 `int count = 0; // init`，AST 會將其解析成樹狀結構，其中 `count` 是一個變數節點 (Variable Node)，`0` 是一個數值節點 (Literal Node)，而 `// init` 則是註解節點 (Comment Node)
- **Tree-sitter:** Tree-sitter 是一個新型的程式碼解析生成工具，他的優點在於傳統解析器在代碼修改後需重新解析整個檔案，而 Tree-sitter 的增

量式解析只需要解析受影響的區塊，不用整個檔案都解析，此外，Tree-sitter 的錯誤恢復機制 讓惡意代碼在不符合標準語法規範情況下，仍然能夠在遇到語法錯誤時跳過損毀部分並繼續構建 CST，是當前程式碼靜態分析的首選工具

- **Concrete Syntax Tree (CST):** 不同於傳統的抽象語法樹 AST，Tree-sitter 所生成的具體語法樹 CST，他保留了原始 code 中的完整 Lexical 語法資訊，包括語法分隔符(if, 標點符號, 括號)，讓模型輸出的 token 機率值能和語法節點進行精確的空間對齊，實現精確地異常檢測
- **Min-K%:** 傳統的困惑度檢測是對所有 token 的 negative log likelihood (loss)取平均值，這會導致短的 adversarial payload 被其他常見字詞的低困惑度給稀釋。然而，Min-K%則是為了解決 Perplexity 缺陷而設計的高階演算法，強迫模型只關注 loss 最高，也就是困惑度最高的片段，這在大量少數惡意 payload 隱藏在大量的正常程式碼下，效果遠優於傳統的 perplexity
- **Logits layer:** 是 Deep Neural Network 架構中的最後一層，會輸出 vocabulary 中每個 token 的原始 unnormalized 預測分數，在堆論過程中，這些原始分數會透過 Softmax 函數轉化為機率分佈，最終用來決定下一個 token 的生成

4. Threat model

A. Problem setting

為了精確描述針對程式碼生成的間接提示注入攻擊(Indirect Prompt Injection Attack)及我們的防禦框架，我們將問題正式定義如下：

Input: 定義輸入為 $x = (P^u, C)$ ，其中 P^u 是開發者輸入的 User Prompt， C 是開發者丟入系統或系統檢索到的外部程式碼，基於程式碼的語法結構，可以將程式碼 C 拆分成兩個互斥的集合：

x_{exec} : 可執行區域 (Executable areas)，包含程式碼的核心運算邏輯，control flow 等等

$x_{non-exec}$: 非執行區域 (Non-executable areas)，攻擊者最容易隱藏惡意 payload，且不影響程式碼編譯的區域，包含註解(D_{com})，字串(D_{str})，及變數名稱(D_{var})，而整體的輸入可表示為 $x = \{P^u, x_{exec}, x_{non-exec}\}$ 。

Model: 將 LLM 定義為一個 mapping function $f(\cdot)$ ，該函數接收輸入 x 並生成預測的惡意程式碼輸出 y

$$f(x) \rightarrow y$$

Attack: 當攻擊者將惡意 payload 隱藏在非執行區域時，存在一個子集

$$S \subseteq x_{non-exec}$$

使得模型在處理包含 S 的上下文時，其輸出會從原本預期的安全輸出轉變成 attacker 所期望的惡意輸出，這表示，移除該惡意子集 S 前後的模型生成結果

將會完全不同，因此，攻擊成功的情境的定義為：

$$f(x) \neq f(x - S)$$

防禦目標(Objectives):

我們防禦框架的核心目標是: 找出最具影響力的最小子集

$$S^* = \arg \max_{S \subseteq x_{non-exec}} \text{dist}(\text{Logits}(f(x)), \text{Logits}(f(x \setminus S)))$$

在非執行區域 $x_{non-exec}$ 中，找到一個能在被移除或中性化後，導致模型最

後一層預測的機率分佈(Logits)產生最劇烈變化的節點子集 S ，代表該子集

對模型生成惡意程式碼具有最大的影響力，即為惡意 Trigger

B. Attacker's capability:

我們假設攻擊者的目標是透過 Indirect 方式來操縱 Code LLM 的輸出，且具備以下能力:

1. 外部 data sources 的操縱: 攻擊者具備將惡意 payload 植入外部程式碼的

能力，這包括將含有惡意 trigger 的程式碼上傳至 GitHub、Stack Overflow

等公開平台，或將其注入到模型檢索到的文檔中，由於模型常將這些平台

作為 RAG 的上下文來源，攻擊者便得以實現 Indirect Prompt Injection，而

不需直接和 model 對話。

2. 設計不同類型 malicious payload 的能力: 攻擊者有能力設計兩種不同特

性的攻擊 payload，首先是 Adversarial Triggers，利用黑箱優化技術生成看

似亂碼的字串，並藉由此字串誘導 LLM 進行指定的惡意輸出；再來是

Semantic Triggers，透過變數重命名和語境重組，製造出在統計和語法上極

為自然的惡意 trigger

3. 黑箱存取權限 (Black-box Access): 攻擊者不需了解目標 LLM 的內部參數

或權重，只需透過公開 API 或網頁介面獲取模型的生成結果，攻擊者能利

用這種黑箱存取權限來測試，優化攻擊 payload 的成功率。

4. 攻擊者的防禦理解(Attacker Knowledge): 為了全面評估防禦框架，這邊

探討了兩種有著不同防禦理解的攻擊者：

- **Standard Attacker:** 攻擊者不知道系統的偵測防禦機制，僅針對目標 LLM 設計出 Indirect Prompt Injection Attack
- **Adaptive Attacker:** 在 Robustness Analysis 中，我們假設攻擊者完全掌握防禦系統的內部細節，他們知道系統使用 Tree-sitter 進行 CST 解析，並且第三層防禦高度依賴 Surprise Score 和 CST 節點的 Influence 機制運作，因此這個 Adaptive Attacker 可以針對性地設計出 Decoys、Copy Trigger、或 Contextual Attack，來試圖繞過我們的防禦偵測系統

C. Defender's capability

我們假設防禦者是程式碼生成系統的開發者或模型部署者，防禦者在系統

中有以下能力：

1. 輸入攔截與審查: 防禦者有權限在所有 prompt 進入 LLM 之前，對其進行攔截，檢查及修改。

2. 靜態分析資源: 防禦者具備足夠權限和資源來運行輕量級的程式碼靜態分析工具 (例如 Tree-sitter)，用來識別程式碼的語法結構。

3. 模型存取權與 Logits 獲取限制: 由於我們的核心防禦機制需要計算

$$\text{dist}(\text{Logits}(f(x)), \text{Logits}(f(x \setminus S)))$$

因此防禦者必須要能取得模型的機率分佈(Logits)及困惑度，然而，許多商業化 LLM API (ex: OpenAI, Anthropic) 不開放存取底層 Logits，因此，我們的防禦框架適用於以下兩種使用情境:

a. 企業本地化部署(On-Premise Deployment):

企業在內部伺服器部署開源 LLM (如 CodeGen, Qwen)，防禦者擁有對目標 LLM 的完整白箱存取權限，可直接獲取 Logits 並整合防禦偵測機制。

b. 雲端黑箱 API:

當開發者必須使用無法取得 Logits 的雲端黑箱 API 模型時 (如 OpenAI)，我們的防禦框架可作為本地端的前置代理，防禦方部署一個輕量級的開源代理模型(Surrogate model, 如 codegen-350M model) 來計算 Logits 變異與萃取惡意 trigger。等防禦系統清潔完惡意程式碼後，再將安全的 Prompt 送入雲端的黑箱 API，確保防禦系統在無法取得目標 LLM Logits 的情境下依然有效

5. Proposed Method

根據 Threat Model 的定義與 Latency Constraint，我們提出了一套 Hierarchical 的 Syntax-Semantic Aware 防禦框架 D_x 目標是針對輸入 $x =$

$\{P^u, x_{exec}, x_{non-exec}\}$ 在確保程式碼執行邏輯 x_{exec} 不受干擾的前提下，以最

低的計算成本精準偵測並清潔惡意 trigger $S \subseteq x_{non-exec}$ 。防禦框架分為兩

個階段：Input Processing & Extraction 以及 Code Guard Detection System

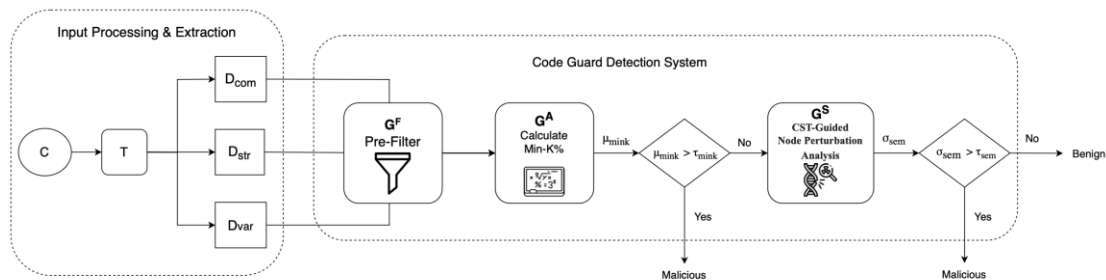


Figure 2 Solution Overview

階段一：輸入解析與目標萃取 (Input Processing & Target Extraction)

當系統接收到外部程式碼 C 時，為避免傳統偵測處理破壞了程式碼功能，

我們先透過 Tree-sitter 工具進行精確的程式碼語法定位：

1. CST 節點構建與程式碼標記: 利用 Tree-sitter (T) 將程式碼 C 解析為

CST，並將其劃分成可執行區域和非執行區域，並保留兩者在原始文本中的

Positional Mapping

2. 高風險目標萃取: 由於攻擊者有更高機率將 payload 藏在影響編譯的非

執行區域

由於攻擊者為了提高 trigger 的隱蔽性以及不影響程式碼編譯，因此有極高

的機率將 payload 藏在程式碼非執行區域中，因此我們將偵測範圍 focus 在

$x_{non-exec}$ 中的註解(D_{com})，字串(D_{str})，及變數名稱(D_{var})中，避免防禦機

制干擾程式碼的核心演算法

階段二: Code Guard Detection System:

我們設計了三層防禦機制，第一層能快速地將明顯的惡意 Prompt 偵測出來，而後續的第二，第三層能夠分別防禦住對抗型攻擊與語意型攻擊

第一層: 輕量級預過濾層 (Syntax-guided Pre-Filtering, G^F):

第一層防禦採用靜態分析技術，並設計出基於 CST 的多維度異常評估機制，目的在於快速過濾出明顯的惡意 Prompt，以減少後續的延遲時間，除了常規的惡意指令匹配之外，也進行以下核心實作:

- a. 引用 Laplace Smoothing 以避免極短的正常變數名稱因為包含特殊字元而被誤判，降低誤判率
- b. 針對 Flashboom 攻擊中利用 Decoy 分散 LLM 注意力的手法，根據函數調用(Call Graph)與共用變數的狀態(State Coupling)建立 interact graph，利用圖的 connected 分析來識別出與主程式無關的 Decoys 區塊

第二層: 基於 CST 與 Entropy 的動態 Min-K% 演算法 (CST-Guided Dynamic Min-K%, G^A):

針對隱藏在 $x_{non-exec}$ 中的對抗型亂碼攻擊 (例如 ShadowCode 與 INSEC)，傳統的 Min-K%演算法通常將整段程式碼視為純文本，並採用固定的 K 值比例，這在面對長度變化極大的程式碼節點時，容易因固定比例 K 而漏判隱藏在其中的短惡意 payload，或在長的正常註解中產生誤判，為解決傳統異常檢測在程式碼領域的侷限性，導入了基於 CST 的動態 Min-K%演算

法，具備以下創新及優化: 我們利用 CST 提供的語法邊界，針對每個候選

節點 $n \in x_{\text{non-exec}}$ 計算其異常得分 μ_{mink}

1. **Dynamic K 與 Entropy 權重評估:** 演算法會根據 CST 節點的類型

(identifier、string、comment) 設定不同的中心長度基準，並引入

Relative Entropy 作為評估指標，當模型對於該節點生成的 Token 機率分

佈呈現極端且混亂時，Entropy 值會讓系統對 K 值進行動態調整，使模

型關注範圍在 Loss 值最高的 Token 區段，有效提升了對隱蔽對抗字串

的敏感度

$$k_{dynamic}(n) = \text{clip}\left(k_{base} \cdot \frac{H(n)}{\log|n|}, 0.05, 0.9\right)$$

2. **多維度異常評分 (Multidimensional Scoring):**

為捕捉隱藏在大量正常文本中的微小對抗性擾動，我們在演算法中結合

了 Sliding window 的平均 loss 與相鄰 token loss 的最大差異值(Max

Spike)，確保攻擊在加入大量正常字元來稀釋整體困惑度情況下，局部

的異常 peak 值仍能被我們的防禦框架偵測出來，計算方式為 給定節點

內的 Token 損失序列 $\{L_1, L_2, \dots, L_{|n|}\}$ ，則最終異常檢測的分數 μ_{mink} 定

義為:

$$\mu_{mink} = \max\left(\text{Mean}(\text{Top } k_{dynamic} \text{ Losses}), \alpha \cdot \text{MaxWindowLoss} + (1 - \alpha) \cdot \text{MaxSpike}\right)$$

此公式包含了全域 Top-K 的 loss 值與 sliding window 的 Max Spike，若

μ_{mink} 大於所設定的 threshold τ_{mink} ，則判定為惡意 trigger

第三層: 基於 CST 引導的節點擾動分析 (CST-Guided Node Perturbation Analysis, G^S):

對於語意型隱蔽攻擊 (例如 XOXO, ITGen)，攻擊者透過常見單字組成的變數名稱或字串來進行攻擊，由於這類攻擊的困惑度極低，無法用第二層的異常檢測抓出，因此，我們在第三層採用了 Differential Mutation Testing 的概念，提出了 CST-Guided Node Perturbation Analysis 概念，量化節點對模型決策的影響力:

1. Surprise-gated Selection:

為了解決節點擾動計算量過大的問題，我們定義 Surprise Score $S(n)$ 作為

前置的過濾條件: $S(n) = \max(L_{contextual}) - L_{prior}$

系統只挑選 $S(n) \geq \tau_{min_surp}$ 的前 k 個節點，作為擾動節點的 candidate，

以此來降低運算開銷

2. Differential Mutation Testing 與影響力量化:

針對候選節點 n ，我們利用 CST 結構執行 Neutral Mutation，我們定義節

點影響力 $I(n)$ 為模型在輸入該節點前後，最後一層 Logits 機率分佈的

Entropy loss 變化量: $I(n) = L(f(x)) - E_{x' \in \text{Mut}(x,n)}[L(f(x'))]$ 其中

$\text{Mut}(x,n)$ 是將節點 n 替換為中性 identifier (ex: VAR 或 TMP) 後的輸入

集合

3. Decision rule 以及 dynamic threshold:

最終判斷節點是否為惡意 Trigger 的 Boolean Function $Trigger(n)$ 定義如下:

$$Trigger(n) = I \left(\left(I(n) > \tau_{dynamic}(n) \right) \vee (Z(n) > \tau_z) \right)$$

其中 $\tau_{dynamic}(n)$ 是基於節點語法類型的自適應 threshold

$$\tau_{dynamic}(n) = \max \left(\tau_{min}, \frac{\tau_{base} \cdot \text{Factor}(n)}{1.0 + S(n) \cdot \tau_{tol}} \right)$$

$\text{Factor}(n)$ 會根據節點類型自動給予權重懲罰，最終區分出正常程式碼與

引導惡意輸出的攻擊 Trigger，最終找出 S

6. Evaluation

A. Setup

i. Datasets

- XOXO
- ShadowCode
- INSEC
- ITGen
- Flashboom
- CoTDeceptor

ii. Models

Used models:

- Salesforce/codegen-350M-mono
- Salesforce/codegen-350M-multi
- Qwen/Qwen3.5-4B
- google/gemma-4-E4B

Victim models:

- Code gemma-2b
- CodeBERT
- GraphBERT
- google/gemma-4-E4B

iii. Metrics

首先說明，偵測被正確偵測為惡意以下四種情況：

1. TP: True Positive, Malicious Prompt 被正確偵測成惡意
2. FP: False Positive, Benign Prompt 被錯誤偵測成惡意
3. TN: True Negative, Benign Prompt 被正確偵測成良性
4. FN: False Negative, Malicious Prompt 被錯誤偵測成良性

則有以下評估指標：

- **Precision:** 被偵測為惡意的 Prompt 中，有多少真的為惡意的

$$\frac{TP}{TP + FP}$$

- **Recall:** 在惡意的 Prompt 當中，有多少是成功被偵測到的

$$\frac{TP}{TP + FN}$$

- **F1 Score:** $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ ，用來確認防禦系統的偵測能力是否足夠好，有沒有保持高準確率以及高召回率

- **FPR:** $\frac{FP}{FP + TN}$ 確認防禦偵測系統的誤判率是否足夠低

- **ASR:** $ASR = \frac{N_{success}}{N_{total}}$ 在所有惡意的 Prompt (包含清理過後的 Prompt) 當中，有多少是能成功攻擊模型的，在 Prompt Injection Defense 的情境中，用來測量防禦清理機制的前後，攻擊成功率是否有顯著下降

- **Utility:** 用來衡量經過清理後的 code，是否有成功保留了原始 code 的行為與品質

iv. Competitors

KillBadCode[6]: 以 Token 為單位，針對想要偵測的 code file，系統會輪流刪除裡面的每個 Token，並計算刪除 token 前後，Cross-entropy 的變化量，如果刪除某個 Token 後，Cross-entropy 有顯著地變化，代表該 Token 的存在讓代碼變得極度不自然（即困惑度高），因此該 Token 就是潛在的 trigger

CodeGarrison[8]: CodeGarrison(CG) 透過設計一個混合的深度學習模型來偵測訓練集中的受污染 data，該模型結合了 Code2Vec (用來捕捉基於 AST 的語法資訊)，CodeBERT (用來理解語意)以及 FastText (用於處理 Token 級別的 variance)，而 CodeGarrison 會在惡意樣本送入模型前進行自動過濾，以確保 data 的安全性

DePA[7]: DePA 提出了一種針對程式碼結構特性設計的 line-level 偵測與清潔方法，與傳統將程式碼視為純文本，只進行 word-level 困惑度分析的 ONION 方法不同，DePA 會計算每一行程式碼在上下文中的困惑度，並將其與檔案整體的困惑度分佈進行比較，若某行困惑度與整體平均值的差值超過所設定的 Threshold T ，則會被判定為潛在的惡意 Trigger 或 Dead Code

v. Hyperparameter Setting

Our Defense Hyperparameter:

每層的核心參數皆透過 Dynamic Threshold Tuning 進行最佳化。以下是各層 Tuning 的關鍵參數及其設定:

- **L1 Pre-Filter, G^F** : 主要針對程式碼的異常統計特徵。調試參數包括最大字數限制 τ_{s1w} (預設 100)、非 ASCII 字元比例 τ_{s1a} (預設 0.001)，以及針對字串、識別碼、註解與語法錯誤的特定 threshold。
- **L2 Adversarial Guardrail, G^A** : 使用 Min-K 複雜度分析檢測對抗性攻擊。核心優化參數為通用對抗閾值 τ_{mink} (預設 10.0) 與針對長字串的專屬閾值 τ_{str} (預設 15.0)。
- **L3 Semantic Guardrail, G^S** : 基於語意影響力的檢測。優化參數包含 Base influence 的 threshold τ_{sem} (預設 0.025)、surprise tolerance $\tau_{sur_{tol}}$ (預設 0.10)、最低 surprise score threshold τ_{sur} (預設 0.15)。我們在實作中透過執行 Grid Search，針對資料集找到最合適的 hyperparameter，讓驗證集取得最高 F1 Score 的參數組合，並作為最終實驗配置。

Baselines:

為確保比較的公平性，各個 Baseline 的核心超參數皆照著其原始論文的設置或針對當前資料集進行最佳化調整:

- **CodeGarrison[8]:** 訓練架構由多層 1D-CNN 與 GRU 組成。模型訓練使用 Adam 優化器，初始 Learning Rate 設為 0.001，批次大小 Batch Size 為 32。訓練過程採用 Cosine Annealing，並加入強度為 10^{-4} 的 L1/L2 正則化與 Dropout (0.3) 以防止 overfitting。訓練 Epoch 設 60，並設 Patience 為 20 的 Early Stopping 以獲取 Validation loss 最低的模型。
- **DePA[7]:** 採用 Qwen3-4B-AWQ 作為檢測模型，最大生成與處理長度 (Max Tokens) 設為 4096。其核心的異常檢測 Anomaly Threshold Multiplier, t 並非固定常數，而是根據不同攻擊資料集的數據分佈進行個別調參 (範圍介於 1.7 至 2.5 之間)，以獲取最佳的 F1-score。
- **KillBadCode[6]:** 我們用 CodeBERT 作為 Tokenizer，並計算序列的 N-gram Cross-Entropy 來評估程式碼的 Naturalness。其餘統計相關參數皆遵循原作者的 default hyperparameter。

B. Result

i. Defense on 6 Attack type compared with 3 baselines

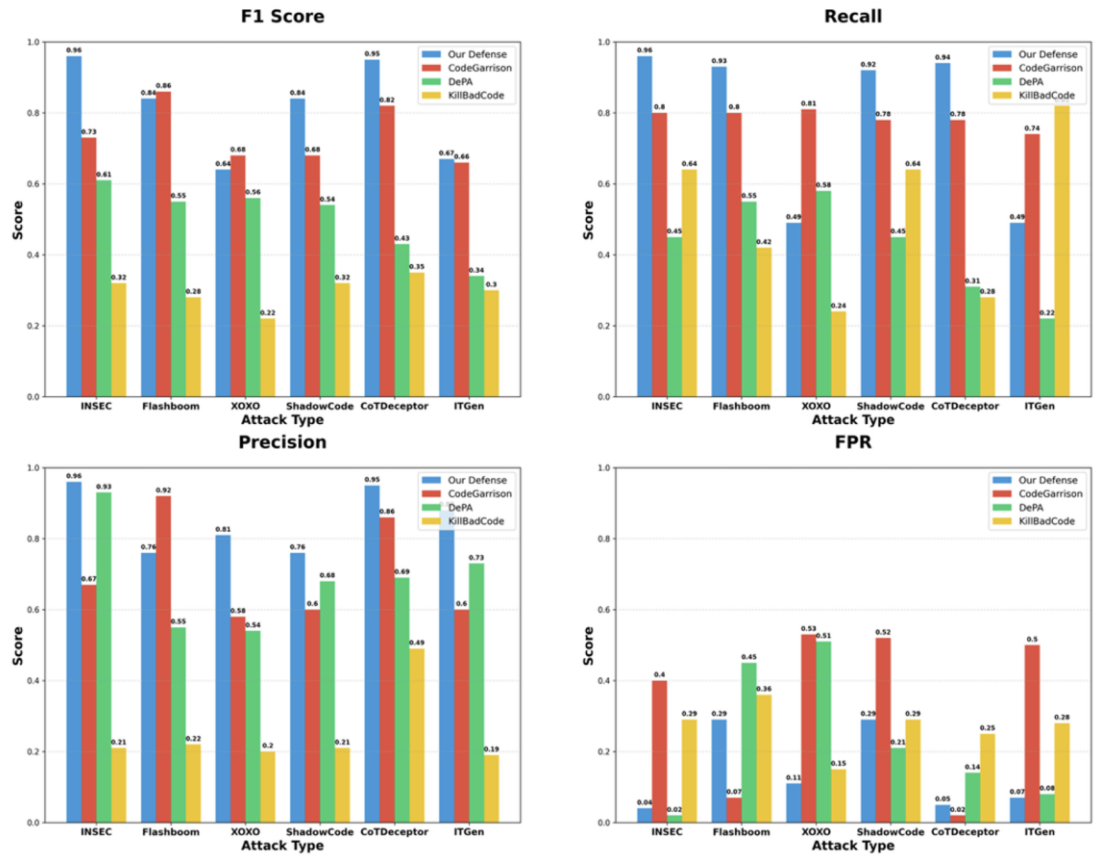


Figure 3. The accuracy of detecting different attack type

以上為我們的防禦框架在面對六種最新的 Prompt Injection Attack 所獲得的偵測結果，與其他三種 Baseline 相比，我們的平均 F1 score 達到 0.817，勝於其他三個 baseline，包括 CodeGarrison [8] (平均 f1 score 為 0.738)、DePA[7] (平均 f1 score 為 0.505)、KillBadCode[6] (平均 f1 score 為 0.28)。

ii. AUC-ROC

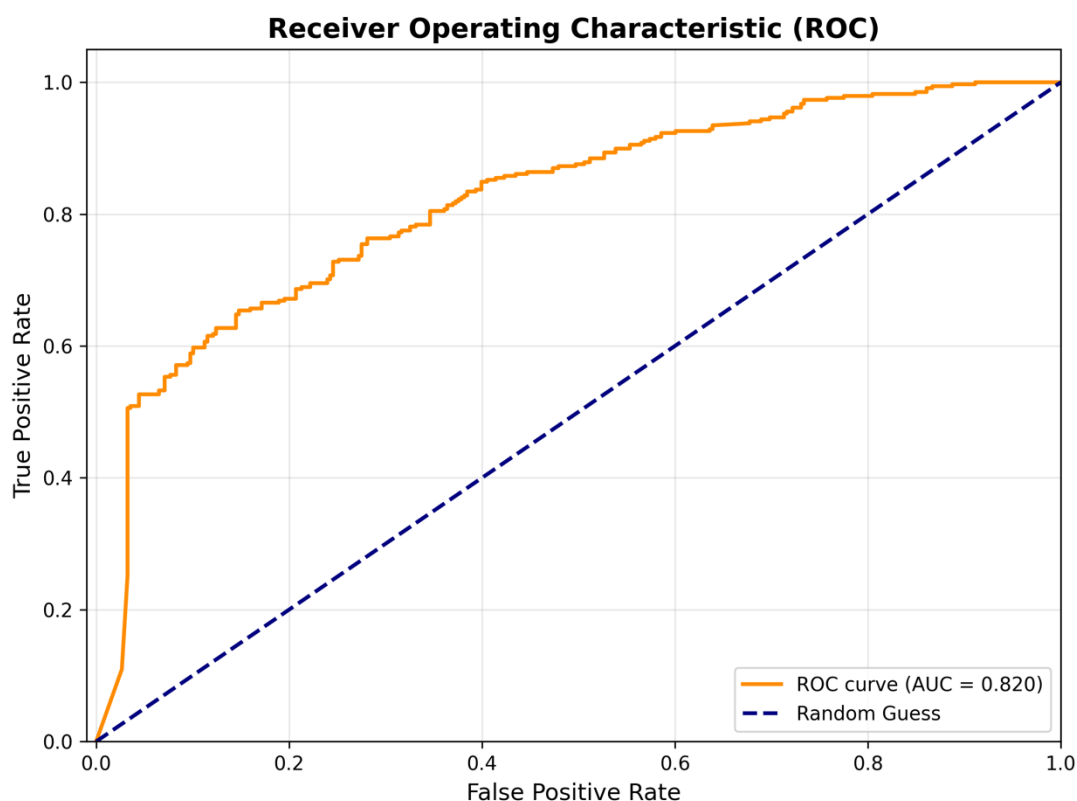


Figure 4. AUC-ROC curve

AUC 0.82: 代表如果隨機挑選一個 Malicious Prompt 和一個 Benign Prompt，我們的防禦系統有 **82%** 的機率能給 Malicious Prompt 更高的威脅評分，這表示在處理 XOXO 或 ITGen 這種隱蔽性攻擊時，有 0.82 的機率可以被偵測出來。

另外可以看到曲線在左下角 (FPR 接近 0 時) 迅速上升，這代表系統能在極低的誤報率下，攔截掉約 50% 的攻擊

iii. Cumulative layer's performance

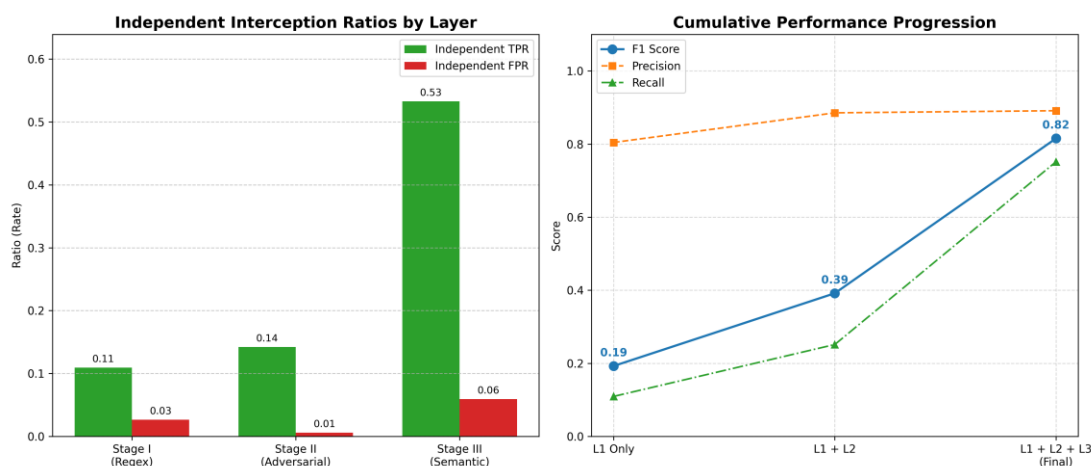


Figure 5. Cumulative layer's performance

左半圖解釋了為什麼第一二層是必須存在的，以及第三層的攔截率是最高的

第一層: 作用在於以極低的計算開銷過濾掉明顯的惡意模式與誘餌攻擊，獨立攔截了 11% 的攻擊，FPR 為 3%。

第二層: 獨立攔截了 14% 的攻擊，證明了異常檢測對於 ShadowCode 以及 INSEC 這類的亂碼型對抗攻擊是有效的。

第三層: 框架的核心，攔截了 53% 的攻擊，顯示出大多數新的隱蔽語意型攻擊需要靠本研究提出的 CST-Guided Node Perturbation Analysis 才能有效識別。

右半圖可解釋各層之間的協同效應

F1 score 從單層的 0.19，加入第二層後提升至 0.39，而在加入第三層後 F1 score 提升至 0.82，然而隨著層數的增加，Precision 皆維持在約 0.88 左右，而 Recall 則大幅成長，這證明了我們的三層架構在提升偵測率的同時，不會引入過多的 False Positive

iv. cross-model generalization

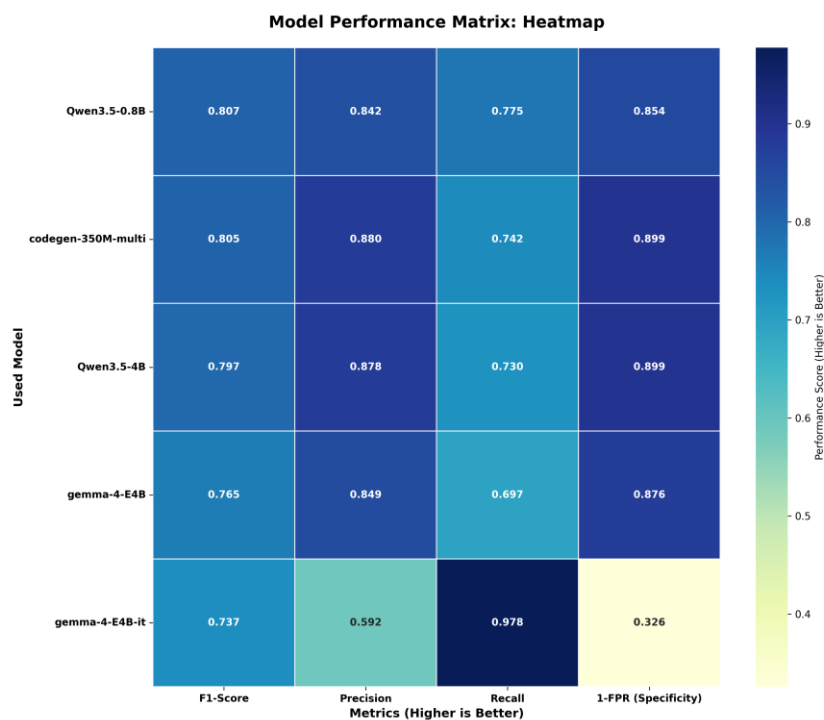


Figure 6. Performance on Different Used model

測試我們的防禦框架能否使用不同的 model 來進行攻擊偵測，結果顯示，我們的防禦框架在使用五種不同的最新開源模型時 (例如 Qwen3.5, gemma-4)，在偵測效能仍維持不錯的成績，在面對六種 Prompt Injection Attack 時平均 F1 score 分別在 0.737 到 0.807 之間

接著，我們也需要測我們的 Victim model 在有無我們的防禦框架情況下，所面對的 Prompt Injection Attack 的 ASR 是否有成功下降，這邊攻擊是六種前面所述的 Prompt Injection Attack 所產生的 ASR，我們的防禦框架所使用的 model 是 Salesforce/codegen-350M-multi 測試的 Victim model 則有: Ministral-3-3B-Base-2512、Mistral-7B-Instruct-v0.3、Qwen3.5-0.8B, gemma-4-E4B、gemma-4-E4B-it

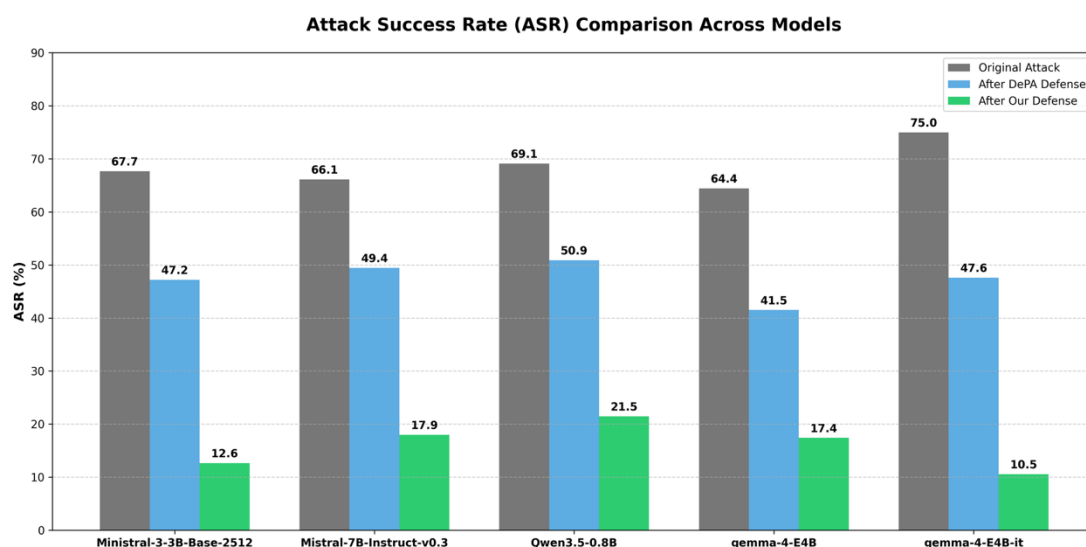


Figure 7. ASR on Different Victim model

從圖 7 可看出，在所有 Victim Models 中，原本 64% - 75% 的攻擊成功率在經過本框架偵測並清潔過後，降低至 10.5% - 21.5% 之間

而不論是 Ministral、Mistral 還是 Qwen，我們的防禦框架在降低 ASR 的表現皆優於 DePA (Baseline) 20% 到 35%，這證明了基於 CST 節點的中性化技術能精確移除惡意 Trigger，而不受限於特定的 Victim 模型

c. Robustness Analysis

1. Adaptive Attack

在此階段，我們針對了防禦系統的強健性進行分析，假設攻擊者在已知我們的防禦框架下去進行的攻擊

為了進一步驗證我們防禦框架的安全性，假設攻擊者知道我們的防禦框架與偵測演算法，設計了以下三種 **adaptive attack**，並針對這些防禦機制進行特定的攻擊優化

- **Decoy Injection:** 針對第三層的 candidate 機制，攻擊者注入 20 個具有 High surprise score 但無惡意節點，來佔用第三層評估的 candidate 名額，進而讓真正的惡意 trigger 不會被第三層偵測到
- **Copy trigger:** 針對第三層所設計，攻擊者在程式碼不同位置注入多個相同的惡意觸發器（如複數個 RAW_QUERIES 變數）。當系統中性化其中一個節點時，剩餘的觸發器仍能維持模型的惡意輸出狀態，導致 Logits 變化不顯著，從而規避偵測
- **Contextual Attack:** 使用 Genetic Algorithm 優化惡意 payload 所在的上下文，降低惡意節點的 surprise score

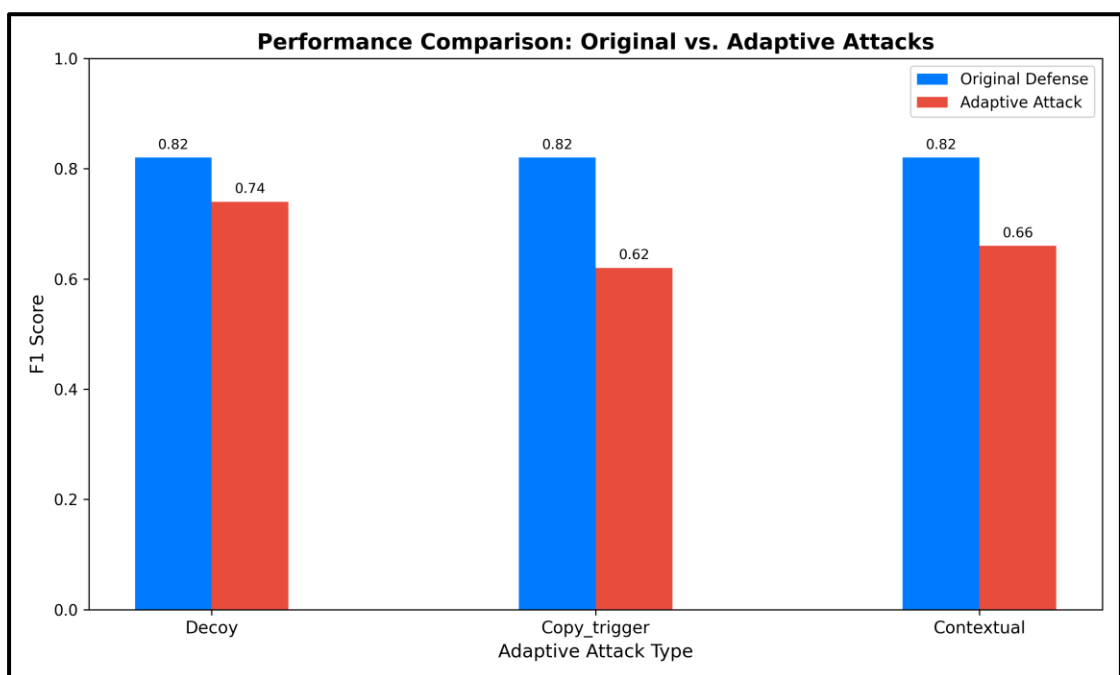


Figure 8. Performance Comparison: Original v.s Adaptive Attacks

面對自適應攻擊，系統的 F1 Score 從原本的 0.82 分別下降至 0.74

(Decoy)、0.62 (Copy_trigger) 與 0.66 (Contextual)。其中，Copy_trigger 對系

統的攻擊滲透最高，顯示在不同地方注入 trigger 能有效地稀釋單一節點擾

動對模型 logits-layer 的影響力，然而，

儘管性能有所下降，本防禦框架仍能維持 0.62 的 F1 Score。遠勝於

KillBadCode (Baseline) 的 0.288 F1 Score

2. Transfer Attack

- **Transfer-based Attack:**

比較攻擊者在 A 模型 (Surrogate) 上生成的惡意 payload，是否能成功攻擊 B

模型 (Victim Model)，以證明該攻擊具有 Transferability (在模型 A 上產生的

惡意 Prompt，也能攻擊到模型 B)，並且看我的防禦框架能不能抵擋這種具

有 Transferability 的攻擊 -> 比較加入我的防禦框架之後，ASR 是否成功降

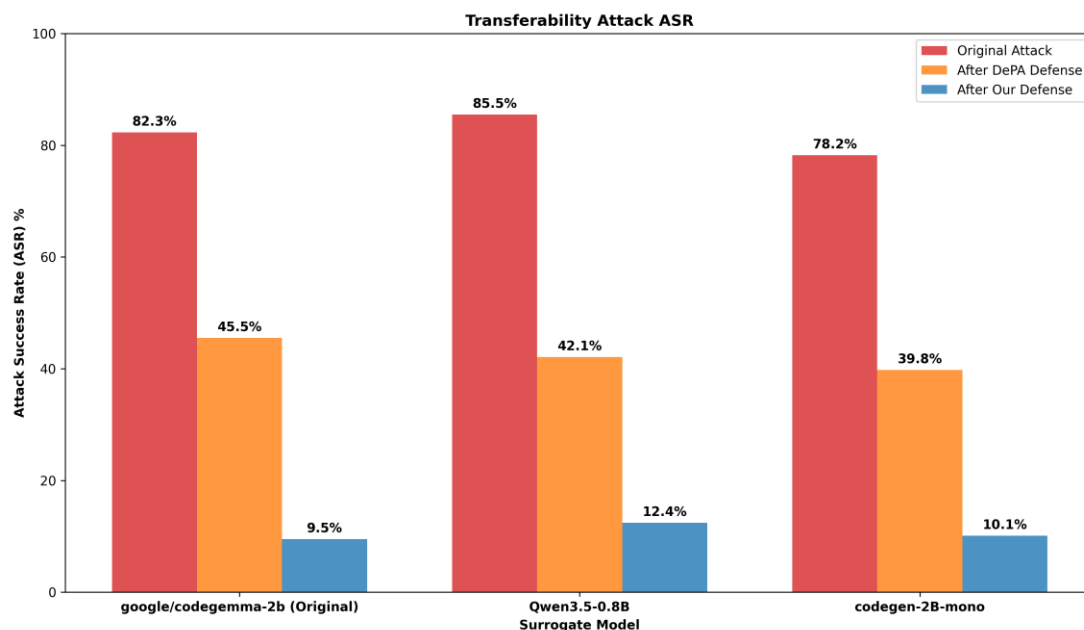
低很多

實驗設置: 在製造 INSEC / ShadowCode attack 時利用 Qwen3.5-0.8B 以及

codegen-2B-mono (Surrogate model) 來找出最佳的惡意 payload，然後將此

payload 餵進 Victim model (gemma-4-E4B-it)

最後在我的防禦框架前後 ASR 變化為:



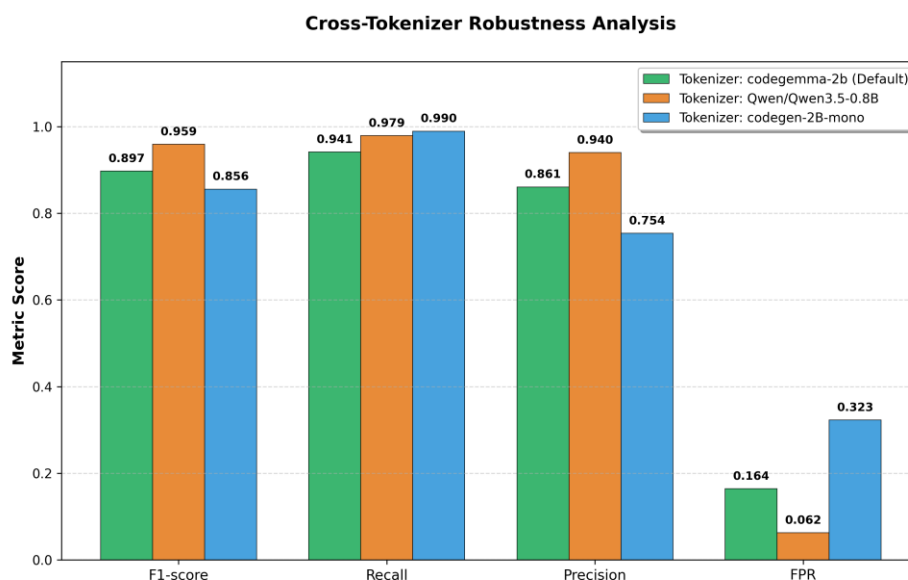
在未加入防禦情況下，攻擊者利用三種不同的 Surrogate models(google/codegemma-2b、Qwen3.5-0.8B、codegen-2B-mono) 生成的惡意內容，成功讓 Victim model 被攻擊，達到高 ASR，證明此攻擊具有強大的 Transferability。而實驗結果證明，我們的防禦框架在面對具有 Transferability 的攻擊時，其防禦清潔效果皆由於 Baseline DePA，並讓 ASR 從原本的 78%~86% 降低至 10%~12% 左右，展現了 Robustness

- **Token Robustness:**

證明我的偵測系統不會對特定的 Tokenization 產生 Overfitting，因為 INSEC / ShadowCode 攻擊很依賴 Token space，如果攻擊者是用 Qwen 的 Tokenizer 優化出的亂碼，而我的防禦系統是用 codegen-350M 的 Logits 與 Tokenizer 來偵測，便可以測試出防禦用的模型是否能辨識出不同 token 空間下的異常

這邊 INSEC / ShadowCode 的 Tokenizer 設定為 Salesforce/codegen-2B-mono

以及 Qwen/Qwen3.5-0.8B 我們的防禦框架 Tokenizer 和 used model 則是用 Salesforce/codegen-350M-multi，以下是結果:



實驗結果顯示，該偵測系統在面對使用不同 Tokenizer，例如

Qwen/Qwen3.5-0.8B 與 codegen-2B-mono 產生的攻擊樣本時，達到了平均

F1 score 0.904 的表現 (介於 0.856~0.959)，證明系統並未對特定的

Tokenization 產生 Overfitting，也證明了我們的防禦系統具備偵測不同語法

與 token 特徵下攻擊行為的能力。

- **針對 Baseline 防禦框架去設計攻擊:**

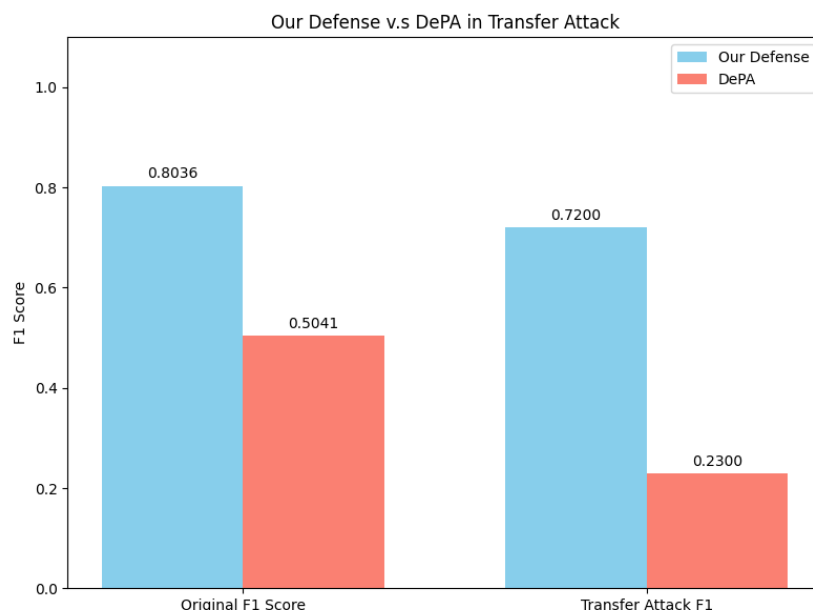
測試我的方法是否在偵測 針對 Baseline 所設計的攻擊，還能擋下來，以此

來證明我比 Baseline 還強

攻擊者利用基因演算法針對 DePA (Baseline)進行優化，攻擊者從 vocab 庫

中找尋 Line-level PPL 極低的 token 組合，來作為自然註解的攻擊 Trigger

以下是 DePA 下降的程度:



Original F1 score 為原本應對最新型攻擊例如 XOXO, ITGen, CoTDeceptor 的平均 F1 Score，可以看到 Baseline DePA 因為攻擊專門針對 DePA 防禦方法而有所下降，但由於該註解是用來誘導 LLM 產生惡意輸出的，所以即便該註解的 Line-level PPL 極低，他在模型的 Logits Layer 仍具有極高的影響力，因此我們的 CST-Guided Node Perturbation Analysis 透過 CST 節點中性化，成功識別出這些自然但具有關鍵影響力的節點，有著 0.72 F1 score 的偵測效能

7. Conclusion

總結來說，本文針對 Code LLM 面臨的 Indirect Prompt Injection Attack, IPIA 提出了一套創新的 Hierarchical Syntax-Semantic-Aware 防禦框架。本研究不僅深入分析了新型攻擊 (如 semantic trigger 與 adversarial perturbation) 的特

徵，更結合了程式碼靜態分析與動態模型決策評估，填補了現有防禦機制

在 code generation 領域的缺口。

本研究的主要貢獻與結論如下：

- **基於程式碼語法的層級化防禦架構:** 不同於傳統將程式碼視為純文本的防禦方法，本框架利用 Tree-sitter 構建 CST，精確鎖定註解、字串與變數名稱等「非執行區域」進行偵測。這種語法感知的層級化設計，確保了在偵測惡意 payload 的同時，不會破壞程式碼的核心執行邏輯與功能，維持了開發工具的實用性。
- **針對對抗性攻擊與語意型攻擊的創新偵測技術:** 本研究針對不同特性的攻擊設計了專屬防禦層。我們改良了傳統的 Min-K% 演算法，引入動態 K 值與熵值 Entropy 權重，有效提升了對隱蔽亂碼型擾動的敏感度。此外，針對隱蔽性極高的 Semantic Trigger，我們提出了創新的 CST-Guided Node Perturbation Analysis，透過量化節點對模型 Logits 影響力的方式，成功偵測出外表自然但具惡意意圖的 Semantic Trigger。
- **Outperforming 的偵測效能與攻擊緩解能力:** 實驗結果顯示，本框架在面對 XOXO、ITGen、ShadowCode 等六種新型 Prompt Injection 攻擊時，平均 F1 Score 達到 0.817，顯著優於 KillBadCode (0.28)、DePA (0.505) 及 CodeGarrison (0.738) 等 Baseline Defense。在清潔攻擊的效果上，我們成

功將 Victim Models 的 ASR 從原本的 64%-75% 降低至 10.5%-21.5% 之間。

- **強大的 Robustness 與 Cross-model generalization:** 本防禦框架展現了優異的泛化能力，在使用不同的開源模型作偵測時，仍能維持穩定的表現。在 robustness analysis 中，即便面對已知防禦機制的 Adaptive Attacker，本系統仍能維持 0.62 的 F1 Score，遠勝於現有的防禦方案。此外，實驗亦證明本框架能有效攔截具備 Transferability 的黑箱攻擊。

綜合以上，本研究提出的防禦框架為 LLM 在程式碼生成領域的安全性提供了一套高效、精確且具 robustness 的解決方案，能偵測並清潔日益複雜的間接提示注入攻擊

Reference

- [1] A. Štorek *et al.*, “XOXO: Stealthy Cross-Origin Context Poisoning Attacks against AI Coding Assistants,” May 20, 2025, *arXiv*: arXiv:2503.14281. doi: 10.48550/arXiv.2503.14281.
- [2] L. Huang, W. Sun, and M. Yan, “Iterative Generation of Adversarial Example for Deep Code Models,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, Apr. 2025, pp. 2213–2224. doi: 10.1109/ICSE55347.2025.00086.
- [3] Y. Yang *et al.*, “ShadowCode: Towards (Automatic) External Prompt Injection Attack against Code LLMs,” Jul. 22, 2025, *arXiv*: arXiv:2407.09164. doi: 10.48550/arXiv.2407.09164.
- [4] S. Jenko, N. Mündler, J. He, M. Vero, and M. Vechev, “Black-Box Adversarial Attacks on LLM-Based Code Completion,” Jun. 13, 2025, *arXiv*: arXiv:2408.02509. doi: 10.48550/arXiv.2408.02509.

- [5] X. Li *et al.*, “Make a Feint to the East While Attacking in the West: Blinding LLM-Based Code Auditors with Flashboom Attacks,” Accessed: Apr. 06, 2026. [Online]. Available: <https://www.computer.org/csdl/proceedings-article/sp/2025/223600a539/26hiTMIdqiA>
- [6] W. Sun *et al.*, “Show Me Your Code! Kill Code Poisoning: A Lightweight Method Based on Code Naturalness,” Feb. 20, 2025, *arXiv*: arXiv:2502.15830. doi: 10.48550/arXiv.2502.15830.
- [7] C.-C. Tsai, C.-M. Yu, Y.-D. Lin, Y.-S. Wu, and W.-B. Lee, “Beyond Natural Language Perplexity: Detecting Dead Code Poisoning in Code Generation Datasets,” Feb. 28, 2025, *arXiv*: arXiv:2502.20246. doi: 10.48550/arXiv.2502.20246.
- [8] E. Ghannoum and M. Ghafari, “Poisoned Source Code Detection in Code Models,” *arXiv.org*. Accessed: May 10, 2026. [Online]. Available: <https://arxiv.org/abs/2502.13459v2>
- [9] H. Li, H. Gao, Z. Zhao, Z. Lin, J. Gao, and X. Li, “LLMs Caught in the Crossfire: Malware Requests and Jailbreak Challenges,” Jun. 09, 2025, *arXiv*: arXiv:2506.10022. doi: 10.48550/arXiv.2506.10022.
- [10] Y. Jiao, X. Wang, and K. Yang, “PR-Attack: Coordinated Prompt-RAG Attacks on Retrieval-Augmented Generation in Large Language Models via Bilevel Optimization,” Jun. 20, 2025, *arXiv*: arXiv:2504.07717. doi: 10.48550/arXiv.2504.07717.
- [11] M. Wahed *et al.*, “MOCHA: Are Code Language Models Robust Against Multi-Turn Malicious Coding Prompts?,” Jul. 25, 2025, *arXiv*: arXiv:2507.19598. doi: 10.48550/arXiv.2507.19598.
- [12] Y. Liu, Y. Zhao, Y. Lyu, T. Zhang, H. Wang, and D. Lo, “‘Your AI, My Shell’: Demystifying Prompt Injection Attacks on Agentic AI Coding Editors,” Sep. 26, 2025, *arXiv*: arXiv:2509.22040. doi: 10.48550/arXiv.2509.22040.
- [13] S. Ouyang, Y. Qin, B. Lin, L. Chen, X. Mao, and S. Wang, “Smoke and Mirrors: Jailbreaking LLM-based Code Generation via Implicit Malicious Prompts,” Mar. 23, 2025, *arXiv*: arXiv:2503.17953. doi: 10.48550/arXiv.2503.17953.
- [14] H. Lin, Y. Lao, T. Geng, T. Yu, and W. Zhao, “UniGuardian: A Unified Defense for Detecting Prompt Injection, Backdoor Attacks and Adversarial Attacks in Large Language Models,” Feb. 18, 2025, *arXiv*: arXiv:2502.13141. doi: 10.48550/arXiv.2502.13141.
- [15] T. Wen *et al.*, “Defending against Indirect Prompt Injection by Instruction Detection,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 19472–19487. doi: 10.18653/v1/2025.findings-emnlp.1060.

- [16] M. Liu *et al.*, “Chasing Moving Targets with Online Self-Play Reinforcement Learning for Safer Language Models,” Oct. 06, 2025, *arXiv*: arXiv:2506.07468. doi: 10.48550/arXiv.2506.07468.
- [17] Y. Chen *et al.*, “Can Indirect Prompt Injection Attacks Be Detected and Removed?,” Oct. 04, 2025, *arXiv*: arXiv:2502.16580. doi: 10.48550/arXiv.2502.16580.
- [18] J. Yi *et al.*, “Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, Jul. 2025, pp. 1809–1820. doi: 10.1145/3690624.3709179.